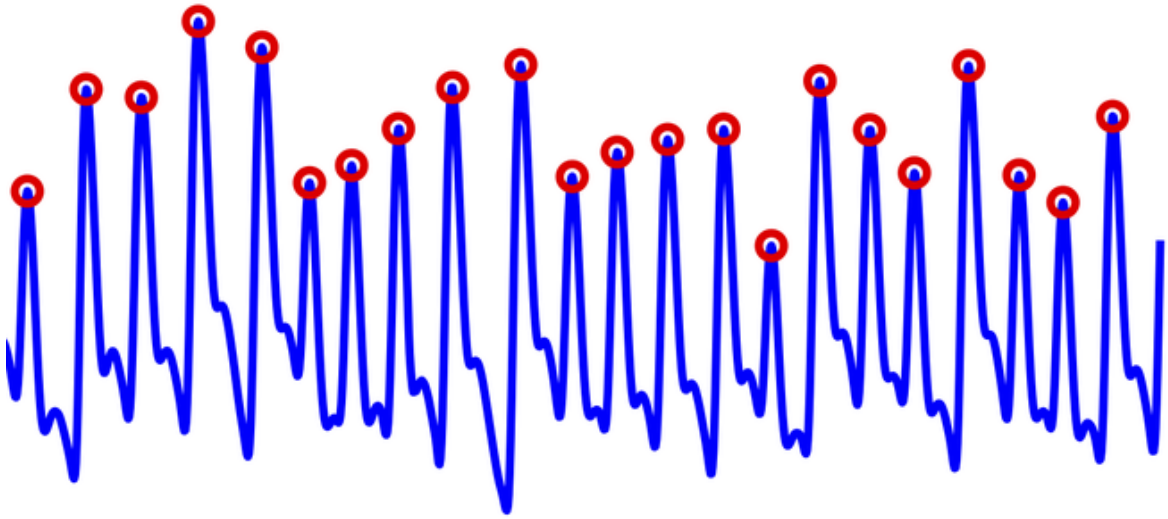
Lab – remote Photoplethysmography



In this lab, you will implement a complete method of heart rate estimation using video analysis. The code structure is provided as well as a video to test your work.

1. Open your IDE, read, understand, and run the code provided. The main file is `main.y`.
   - Function `getTraceFromVidFile()` estimates the RGB traces from the video file (based on a face detector and tracker).
   - Function `getPulseSignalFromTrace()` is supposed to estimate the pulse rPPG trace from the RGB trace.
   - Function `getHRFromPulse()` will estimate HR from the pulse signal.

2. Complete the `getPulseSignalFromTrace()` function. This function will load the RGB traces in *rgbTraces.mat*, the ground truth in *gtdump.xmp* and estimate the rPPG pulse trace from the RGB traces. You will apply band pass filtering and normalization on the Green channel of your RGB traces and save the result in a *pulseTrace.mat* file. Choose the cut-off frequencies of your filter carefully!

3. Complete the `getHRFromPulse()` function. This function should load the rPPG pulse trace in *pulseTrace.mat*, load the PPG pulse trace (obtained by a pulse oximeter, i.e. the ground truth) and then estimate the Heart Rate with a sliding window. For each position of this sliding window, you will estimate HR based on the FFT of the pulse signal. You will do this procedure on the PPG signal and the rPPG signal. It is important to plot the different signals to check that everything is working nicely. The output of the function is 2 vectors: one vector that contains the sequence of HR values estimated from the PPG signal and the sequence of HR values estimated from rPPG signal.

yannick.benezeth@u-bourgogne.fr

4. Now, in the main file, you will plot estimated HR values to compare your estimations and the ground truth.

You can also calculate the Mean Absolute Error (MAE) to have a precise evaluation of the performance of your first method.

5. Then, you will gradually improve the performance of your algorithm. You can first modify the function `getPulseSignalFromTrace()` and estimate the difference between the green and red channels:

$$rppg(t) = G(t) - R(t)$$

6. You will now implement the method named Chrom[1]. Take few minutes to read and understand this paper quickly. To implement this method, you will need to:
   a. Modify your normalization method and implement their normalization method described in equation (2). The average μ(Ci) is calculated on a 1 second signal.
   b. Then implement equation (16) on a new sliding window on the RGB traces. This sliding window is quite small, you can use 1 second for example with a step of 0.5 second between each window.
   c. Then, plot the results to validate your improvements.

7. If you have time, you can also improve the estimation of the RGB trace. To do this, you can use a skin pixel detection to avoid considering the background pixels in the calculation of the spatial average at each frame. Update the `getTraceFromVidFile()` function to add the skin pixel detection.

8. Eventually, if you still have time, you can download the UBFC-rPPG dataset here. You can then process all the video files and objectively compare your methods.

[1] De Haan, G., & Jeanne, V. (2013). Robust pulse rate from chrominance-based rPPG. *IEEE Transactions on Biomedical Engineering*, *60*(10), 2878-2886.